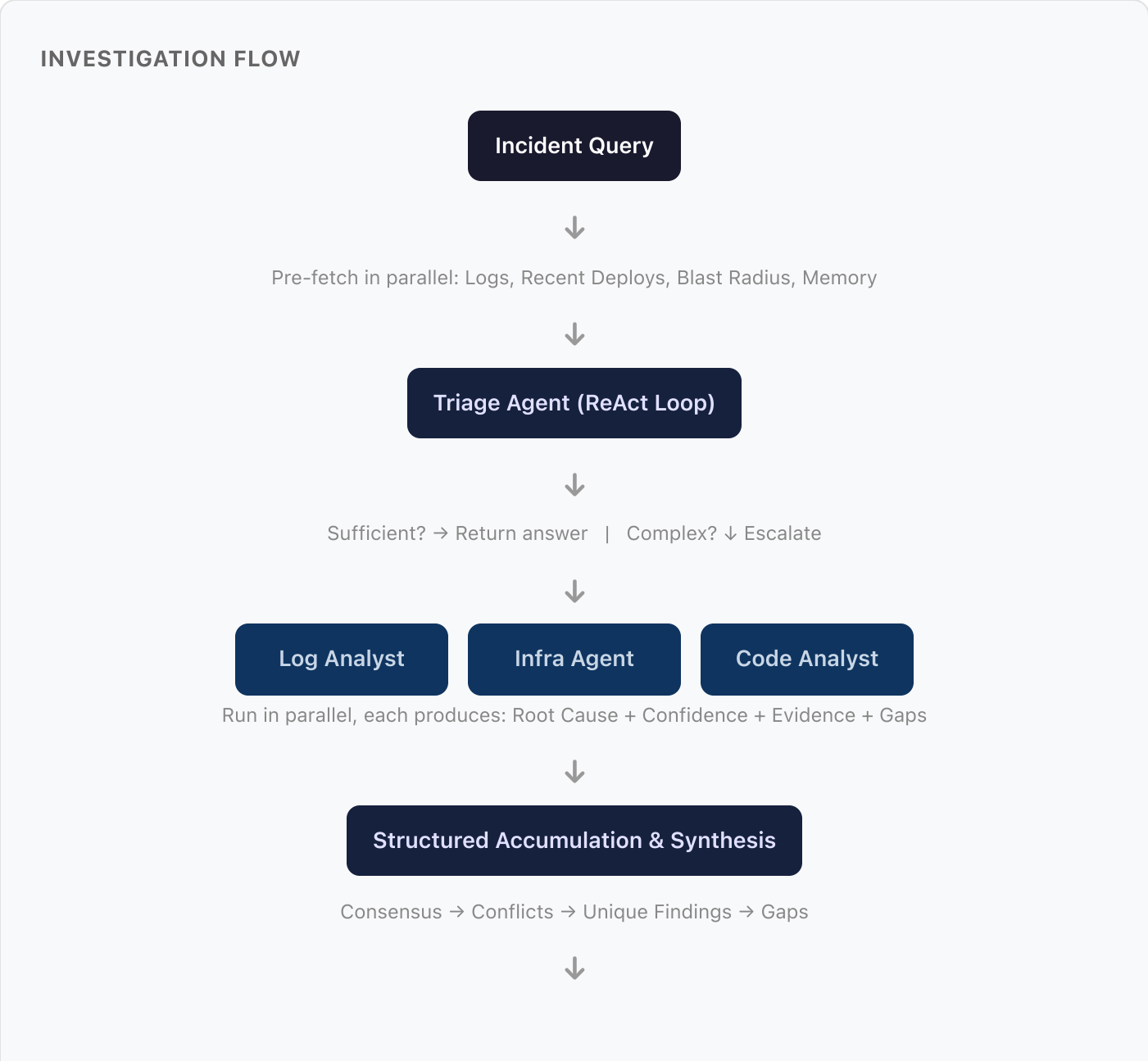
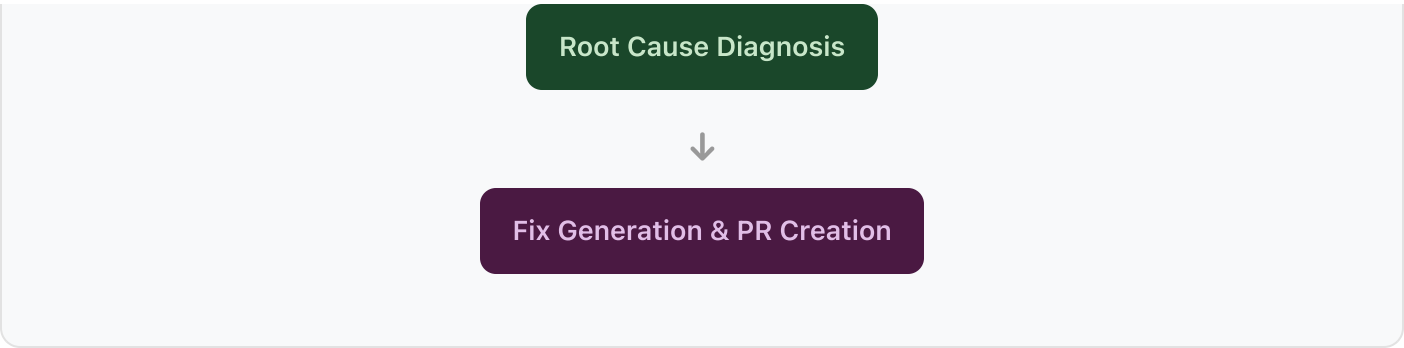
AI SRE Agent — From Alert to Root Cause to Fix. Built It, Shipped It, My Team Uses It Every Day.

Problem

When a production incident hits, SREs spend 30-60 minutes manually piecing together the picture — jumping between Grafana logs, AWS dashboards, database queries, source code, and recent deploy history. The diagnosis depends on tribal knowledge that's hard to transfer, and on-call engineers often lack context for services they didn't build. I wanted to automate this entire investigation workflow, not just the "summarize these logs" part.





What I Built

End-to-end autonomous investigation pipeline — The agent takes a natural language query ("what happened to request abc-123 in payment-service?"), queries live production systems (Grafana, AWS, PostgreSQL, Sentry, GitHub), traces the data flow through code, and produces a root cause diagnosis with cited evidence. Follow-up questions maintain full conversation context.

Plugin architecture (SPI pattern) for zero-lock-in integrations — Every external system (monitoring, cloud, database, code hosting, error tracking) is behind a Service Provider Interface. Swapping Grafana for Datadog or AWS for GCP means implementing one interface — the agents and core logic don't change. Currently 5 plugins in production: Grafana (Loki + Prometheus), GitHub, AWS (ECS, ALB, SQS, CloudWatch), PostgreSQL, and Sentry.

Multi-agent system with structured accumulation — A fast triage agent handles most incidents alone using a ReAct loop (reason → call tools → observe → reason again). For complex cases — multi-service outages, conflicting signals — it auto-escalates to specialist agents (log analyst, infrastructure, code & data flow) that run in parallel. Each specialist produces structured findings (root cause, confidence score, key evidence, investigation gaps) and a synthesis agent merges them — identifying consensus, resolving conflicts, and surfacing what only one specialist caught.

Custom agents & skills — extensible without code changes — Teams onboard their own specialist agents via API: define a system prompt, assign plugins and skills, pick an LLM model, set token budgets. The orchestrator automatically considers custom agents alongside built-in ones when deciding who to dispatch. For skills, existing operations runbooks written as markdown (with embedded bash, SQL, PromQL blocks) become executable agent capabilities. Register a "check Kafka consumer lag" playbook and any agent can invoke it. Domain-specific tribal knowledge gets codified once and is available to every agent going forward.

Memory system that learns from corrections — Three-tier memory backed by pgvector: episodic (past incidents and outcomes), semantic (system topology, baselines), procedural (correction rules). When a human says "your diagnosis was wrong, the actual cause was connection pool exhaustion" — the system creates a rule that gets injected into future investigations. The agent doesn't make the same mistake twice.

Autonomous fix generation & PR creation — Once the root cause is confirmed, the agent reads the actual source files from GitHub, generates a minimal code patch, creates a branch, commits the fix, and opens a pull request — the LLM decides the entire flow autonomously using individual tools (read file, create branch, commit with SHA, create PR) rather than a rigid scripted sequence.

Real-time streaming with reasoning trail — SSE endpoints stream the agent's thought process live: which tools it's calling, what it found, when it's escalating to specialists. The team sees the investigation unfold in real-time instead of waiting for a black-box answer.

Dissatisfaction-driven escalation — If a user says "that's wrong" or "dig deeper" on a follow-up, the system detects the sentiment and automatically dispatches specialist agents for a deeper investigation.

Impact

My team uses this daily for production incident triage. What used to take 30-60 minutes of manual investigation now gets a first-pass diagnosis with cited evidence from live systems. The team has onboarded their own custom skills — service-specific runbooks and database diagnostic queries — so the agent's knowledge base keeps growing with our domain expertise. The platform is 14 modules, 112 source files, 5 plugin implementations, 6 built-in agents, 25+ tool methods, and 12 externalized prompt files — designed as a platform teams extend, not a point solution they outgrow.

Tools & Technologies Used

LLMs — Anthropic Claude (Sonnet for deep reasoning, Haiku for fast triage) — used for

autonomous tool-calling agents via ReAct pattern, not just prompt-response

AI Framework — Spring AI 1.0 — ChatClient, function/tool calling, structured output

Embeddings — OpenAI Embeddings API — for skill discovery and memory similarity search

Vector Search — PostgreSQL + pgvector — stores incident memory and skill embeddings, no separate vector DB needed

Agent Orchestration — Custom-built multi-agent framework — agent registry, dynamic tool binding per agent based on assigned plugins, parallel specialist dispatch with structured accumulation

Cloud Platform — AWS (ECS, ALB, SQS, CloudWatch, Health API) — both as a diagnostic target the agent queries and as the deployment platform

Monitoring — Grafana (Loki for logs, Prometheus for metrics) — the agent queries these live during investigation

Error Tracking — Sentry — error groups, stack traces, release health correlation

Code Hosting — GitHub API — source code analysis, git history, autonomous PR creation

Database Diagnostics — PostgreSQL — read-only diagnostic queries (pg_stat_activity, pg_stat_statements, lock detection, replication lag)

Backend — Java 21, Spring Boot 3.3, Spring Security, JPA/Hibernate

Data — PostgreSQL 16, Redis, Flyway migrations

Streaming — Server-Sent Events (SSE) for real-time reasoning trail

Infrastructure — Docker, Docker Compose

Architecture Patterns — Plugin SPI, ReAct agent loop, fan-out/fan-in multi-agent, structured accumulation, ThreadLocal event streaming, markdown-as-code skill engine